

Работа с GPIO в Linux в режиме ШИМ

Владислав Богданов

2026-06-05

Оглавление

1. Введение	1
2. Меняем Pinmux (GPIO → PWM)	1
3. Ищем нужный шим контроллер	1
4. Пошаговая настройка ШИМ	3
4.1. Результаты	4
4.1.1. Стандартный интерфейс Linux (sysfs)	4
4.1.2. Продвинутые режимы (Аппаратные возможности)	4
4.1.3. Главная проблема sysfs и современное решение	5

1. Введение

И так сначала мы занимались ногодрыгом (gpio_output.pdf), достучались до пинов. Далее создали файл инициализирующий его режим gpio при загрузке системы.

2. Меняем Pinmux (GPIO → PWM)

Вспомним таблицу из даташита.

table 10.35 – continued from previous page

Pin Num	Pin Name	Function_select_register	fmux default	Description
26	JTAG_CPU_TMS_TMS	FMUX_GPIO_R EG_IOCTLL_JTAG_CPU_TMS 0x0300_1064	0x0	IO JTAG_CPU_TMS function select: • 0 : CR_4WTMS (default) • 1 : CAM_MCLK0 • 2 : PWM[7] • 3 : XGPIOA[19] • 4 : UART1_RTS • 5 : AUX0 • 6 : UART1_TX • 7 : VO_D[28] • Others : Reserved
27	JTAG_CPU_TCK	FMUX_GPIO_R EG_IOCTLL_JTAG_CPU_TCK	0x0	IO JTAG_CPU_TCK function select:

Для пина (PA19) по адресу 0x03001064: 3 = XGPIOA[19] (то, что мы делали) 2 = PWM[7] (то, что нам нужно сейчас!)

Откроем скрипт инициализации (/etc/init.d/S99...) и меняем значение в команде devmem:

```
# Было:
devmem 0x03001064 32 3
# Стало:
devmem 0x03001064 32 2
```



- Строки с echo 499 > .../export и настройкой направления out можно закомментировать или удалить, они больше не нужны и могут даже вызвать ошибку, так как пин больше не GPIO.
- Примени изменения: перезагрузи плату (reboot) или просто выполни команду `devmem 0x03001064 32 2` прямо в консоли.

3. Ищем нужный шим контроллер

В Linux PWM-устройства доступны через sysfs по пути /sys/class/pwm/. Но нам нужно узнать, под каким номером (pwmchipX) и с каким каналом (pwmY) зарегистрирован именно PWM7 на SG2002.

1. Выполняем:

```
# ls -l /sys/class/pwm/
total 0
lrwxrwxrwx    1 root    root          0 Jun  8 13:58 pwmchip0 ->
../../devices/platform/3060000.pwm/pwm/pwmchip0
lrwxrwxrwx    1 root    root          0 Jun  8 13:58 pwmchip12 ->
../../devices/platform/3063000.pwm/pwm/pwmchip12
lrwxrwxrwx    1 root    root          0 Jun  8 13:58 pwmchip4 ->
../../devices/platform/3061000.pwm/pwm/pwmchip4
lrwxrwxrwx    1 root    root          0 Jun  8 13:58 pwmchip8 ->
../../devices/platform/3062000.pwm/pwm/pwmchip8
```

то есть в описании на процессор мы видим 4 контроллера ШИМ с адресами похожими на данные строки.

Нам нужно "заглянуть" внутрь каждого, чтобы найти тот, у которого есть 7-й канал (или который сам является 7-м).

3.2 Address-Space Mapping

Table 3.4: Memory mapping

Start Address [31:0]	End Address [31:0]	Function Description	Size (Byte)
0x01000000	0x017FFFFF	Reserved	8M
0x01800000	0x018FFFFF	Reserved	
0x01900000	0x01900FFF	ap_mailbox	4K
0x01901000	0x01901FFF	ap_system_ctrl	4K
0x01902000	0x0190EFFF	Reserved	
0x01F00000	0x01F0FFFF	Reserved	64K
0x01F10000	0x01FFFFF	Reserved	
0x02000000	0x02FFFFFF	Reserved	64K
0x03000000	0x03000FFF	TOP_MISC	4K
0x03001000	0x03001FFF	PINMUX	4K
0x03002000	0x03002FFF	CLKGEN/PLL	4K
0x03003000	0x03003FFF	RSTGEN	4K
0x03004000	0x03005FFF	Reserved	
0x03006000	0x03006FFF	Reserved	4K
0x03007000	0x03008FFF	Reserved	
0x03009000	0x03009FFF	Reserved	4K
0x0300A000	0x0300AFFF	Reserved	4K
0x0300B000	0x0300BFFF	Reserved	
0x03010000	0x03010FFF	WATCH DOG0	4K
0x03011000	0x03011FFF	WATCH DOG1	4K
0x03012000	0x03012FFF	WATCH DOG2	4K
0x03020000	0x03020FFF	GPIO0	4K
0x03021000	0x03021FFF	GPIO1	4K
0x03022000	0x03022FFF	GPIO2	4K
0x03023000	0x03023FFF	GPIO3	4K
0x03024000	0x03024FFF	Reserved	
0x03030000	0x03030FFF	WGN0	4K
0x03031000	0x03031FFF	WGN1	4K
0x03032000	0x03032FFF	WGN2	4K
0x03033000	0x03033FFF	Reserved	
0x03040000	0x0304FFFF	KEYSCAN	64K
0x03050000	0x0305FFFF	EFUSE	64K
0x03060000	0x03060FFF	PWM0	4K
0x03061000	0x03061FFF	PWM1	4K
0x03062000	0x03062FFF	PWM2	4K
0x03063000	0x03063FFF	PWM3	4K
0x03064000	0x0309FFFF	Reserved	
0x030A0000	0x030AFFFF	TIMER	64K
0x030C0000	0x030CFFFF	Reserved	
0x030D0000	0x030D0FFF	Reserved	4K
0x030D1000	0x030D1FFF	Reserved	4K
0x030D2000	0x030D2FFF	Reserved	4K
0x030D3000	0x030DFFFF	Reserved	
0x030E0000	0x030EFFFF	TEMPSEN	64K
0x030F0000	0x030FFFFF	SARADC	64K
0x04000000	0x0400FFFF	I2C0	64K

continues on next page

2. Выполняем:

```
# cat /sys/class/pwm/pwmchip0/npwm
4
# cat /sys/class/pwm/pwmchip4/npwm
4
```

```
# cat /sys/class/pwm/pwmchip8/npwm
4
# cat /sys/class/pwm/pwmchip12/npwm
4
```



Эта команда покажет, сколько каналов поддерживает этот чип. Если, например, 8, то каналы будут от 0 до 7.

В нашем случае 4.

3. Получаем

- Названия чипов: pwmchip0, pwmchip4, pwmchip8, pwmchip12.
- Количество каналов у каждого: npwm = 4.



В Linux для таких контроллеров цифра в названии pwmchipX означает базовый (стартовый) номер канала для этого чипа.

Расклад по каналам получается такой:

- pwmchip0: каналы 0, 1, 2, 3
- pwmchip4: каналы 4, 5, 6, 7 ← Вот он!
- pwmchip8: каналы 8, 9, 10, 11
- pwmchip12: каналы 12, 13, 14, 15

То есть нам нужно работать с каналом 3 на pwmchip4.

4. Пошаговая настройка ШИМ

Убеждаемся, что в предыдущем шаге переключили пин PA19 в режим PWM (значение 2 по адресу 0x03001064). Если нет — делаем это сейчас.

Затем выполняем следующие команды в консоли:

1. Экспортируем 3-й канал чипа pwmchip4:

```
echo 3 > /sys/class/pwm/pwmchip4/export
```



Если всё хорошо, появится папка pwm3 внутри pwmchip4. Если выдаст ошибку "Device or resource busy", значит канал уже кем-то занят, но это редкость

```
# ls /sys/class/pwm/pwmchip4/
device      export      npwm        pwm3        subsystem  uevent
unexport
```

2. Настраиваем период (Частота 1 кГц = 1 000 000 наносекунд):

```
echo 1000000 > /sys/class/pwm/pwmchip4/pwm3/period
```



Период может быть любой точное значение определяем с учетом ограничений железа, то есть смотрим регистры.

3. Настраиваем скважность (50% = 500 000 наносекунд):

```
echo 500000 > /sys/class/pwm/pwmchip4/pwm3/duty_cycle
```



Не забываем скважность от 0 до значения периода

4. Включаем генерацию сигнала:

```
echo 1 > /sys/class/pwm/pwmchip4/pwm3/enable
```

5. Важный момент: полярность (Polarity)

```
# Нормальный сигнал (высокий уровень = активный)
echo normal > /sys/class/pwm/pwmchip4/pwm3/polarity

# Инвертированный (низкий уровень = активный)
echo inversed > /sys/class/pwm/pwmchip4/pwm3/polarity
```

4.1. Результаты

4.1.1. Стандартный интерфейс Linux (sysfs)

В стандартном интерфейсе `/sys/class/pwm/pwmchipX/pwmY/` есть всего 4 параметра:

- **period** — Период (частота).
- **duty_cycle** — Длительность импульса (скважность).
- **polarity** — Полярность (normal или inversed).
- **enable** — Включение/выключение (1 или 0).

На этом возможности стандартного sysfs заканчиваются. Если нужно сделать что-то сложнее, упираемся в ограничения этого интерфейса.

4.1.2. Продвинутое режимы (Аппаратные возможности)

Сам контроллер PWM внутри SG2002 (и большинства современных SoC) умеет гораздо больше. Но эти режимы обычно не выведены в простой sysfs, потому что они специфичны

для конкретных задач.

Вот что бывает в "профессиональном" PWM:

- **Выравнивание по центру (Center-aligned PWM)**

- **Что это:** В обычном режиме (Edge-aligned) импульс начинается сразу после начала периода. В режиме Center-aligned импульс находится ровно по центру периода.
- **Зачем нужно:** Критически важно для управления электродвигателями (FOC управление). Это снижает гармонические искажения и нагрев мотора.
- **Как включить в Linux:** В стандартном sysfs этой настройки нет. Обычно это делается через Device Tree или специфичные для вендора (Sophgo) утилиты.

- **Мертвое время (Dead-time insertion)**

- **Что это:** Когда у тебя есть два ШИМ сигнала (один прямой, другой инверсный) для управления H-мостом (управление мотором в обе стороны), нужно гарантировать, что они никогда не будут открыты одновременно. Иначе будет короткое замыкание (shoot-through). Dead-time — это микрораздержка между выключением одного транзистора и включением другого.
- **Зачем нужно:** Спасает силовые ключи от сгорания.
- **Как включить:** Только через прямую запись в регистры (как мы делали с devmem) или через специальный ioc1 драйвера.

- **Фазовый сдвиг (Phase Shift)**

- **Что это:** Возможность запустить несколько PWM каналов с одинаковой частотой, но со сдвигом по фазе (например, канал 1 стартует на 0°, канал 2 на 90°, канал 3 на 180°).
- **Зачем нужно:** В многофазных преобразователях питания или для управления шаговыми двигателями.

4.1.3. Главная проблема sysfs и современное решение

Мы настраиваем ШИМ тремя разными командами echo.

```
echo 1000000 > period
echo 500000 > duty_cycle
echo 1 > enable
```

В чем проблема?

Между выполнением первой и второй команды проходит несколько миллисекунд. В этот момент на пине может появиться глитч (неправильный импульс), потому что ядро применило новый период, но еще не успело применить новую скважность. Для точной электроники это недопустимо.

Решение: Character Device API (/dev/pwmchipX)

Сейчас в Linux уходит от sysfs к символьным устройствам. Это позволяет делать атомарные

обновления (изменять период и скважность одной командой, без глитчей).

Проверим, есть ли он у тебя:

```
ls -l /dev/pwmchip4
```

Если файл есть, значит, ядро поддерживает современный API.

Если ты будешь писать программу на C/C++ для управления ШИМ, никогда **не используй sysfs (открытие файлов и echo)**. Используй библиотеку **libgpiod** (в новых версиях она поддерживает PWM) или прямые вызовы `ioctl` к `/dev/pwmchipX`. Это стандарт индустрии.